

SQL Joins Notes

CROSS JOIN

Definition

A CROSS JOIN returns the Cartesian Product of two tables.

Every row in the first table is paired with every row in the second table.

If

Table A has m rows

Table B has n rows

Result contains $m \times n$ rows.

Syntax

Explicit syntax

```
SELECT *  
FROM Students  
CROSS JOIN Courses;
```

Real Uses

1. Calendar Generation

Dates $\times$ Employees to generate attendance sheets.

SELF JOIN

One of the most important interview joins.

Definition

A table is joined with itself.

Since SQL cannot distinguish between the two copies, aliases are required.

Syntax

```
SELECT ...  
FROM Employee e  
JOIN Employee m  
ON e.manager_id = m.employee_id;
```

Here

Employee → Employee (employees)

Employee → Employee (managers)

Same table. Different roles.

Example 1 — Employee → Manager

Employee

id	name	manager_id
1	Arpit	NULL
2	Aadz	1
3	Riya	1

Query

```
SELECT  
    e.name AS employee,  
    m.name AS manager  
FROM Employee e  
LEFT JOIN Employee m  
ON e.manager_id = m.id;
```

Output

employee	manager
Arpit	NULL
Aadz	Arpit
Riya	Arpit

LeetCode Questions

1. 181. Employees Earning More Than Their Managers ★

Very famous.

Employee

id | name | salary | managerId

Need employees whose salary exceeds their manager's.

Solution

```
SELECT
    e.name AS Employee
FROM Employee e
JOIN Employee m
ON e.managerId = m.id
WHERE e.salary > m.salary;
```

2. 197. Rising Temperature ★

Although not an employee table, it's another SELF JOIN pattern.

Need today's temperature greater than yesterday.

```
SELECT
    w1.id
FROM Weather w1
JOIN Weather w2
ON DATEDIFF(w1.recordDate, w2.recordDate) = 1
WHERE w1.temperature > w2.temperature;
```

NATURAL JOIN

Very rarely used in production.

Interviewers ask it mainly to test SQL knowledge.

Definition

Instead of writing

```
ON table1.column = table2.column
```

SQL automatically joins using all columns that have the same name.

Example

Students

| student_id | name |

Enrollments

| student_id | course |

Query

```
SELECT *  
FROM Students  
NATURAL JOIN Enrollments;
```

Equivalent to

```
SELECT *  
FROM Students  
INNER JOIN Enrollments  
ON Students.student_id = Enrollments.student_id;
```

Advanced Problem Patterns

Managers with at Least 5 Direct Reports

Solution (Self Join)

```
SELECT
    m.name
FROM Employee e
JOIN Employee m
ON e.managerId = m.id
GROUP BY m.id, m.name
HAVING COUNT(*) >= 5;
```

LeetCode 626 — Exchange Seats (Adjacent Self Join Pattern)

Solution

```
SELECT
    s1.id,
    COALESCE(s2.student, s1.student) AS student
FROM Seat s1
LEFT JOIN Seat s2
ON (
    (s1.id % 2 = 1 AND s2.id = s1.id + 1)
    OR
    (s1.id % 2 = 0 AND s2.id = s1.id - 1)
)
ORDER BY s1.id;
```

LeetCode 601 — Human Traffic of Stadium (Self Join Approach)

Solution

```
SELECT DISTINCT s1.*
FROM Stadium s1
JOIN Stadium s2
ON ABS(s1.id - s2.id) = 1
JOIN Stadium s3
ON ABS(s2.id - s3.id) = 1
WHERE s1.people >= 100
    AND s2.people >= 100
    AND s3.people >= 100
ORDER BY s1.visit_date;
```

LeetCode 180 — Consecutive Numbers (Self Join Approach)

Solution

```
SELECT DISTINCT l1.num AS ConsecutiveNums
FROM Logs l1
JOIN Logs l2
ON l1.id + 1 = l2.id
JOIN Logs l3
ON l2.id + 1 = l3.id
WHERE l1.num = l2.num
      AND l2.num = l3.num;
```

Triangle Judgement (Self-Comparison Style)

Solution

```
SELECT
    x,
    y,
    z,
    CASE
        WHEN x + y > z
            AND y + z > x
            AND x + z > y
        THEN 'Yes'
        ELSE 'No'
    END AS triangle
FROM Triangle;
```